

Feature Extraction from Audio Signal with librosa

We follow <https://www.kdnuggets.com/2020/02/audio-data-analysis-deep-learning-python-part-1.html>
(<https://www.kdnuggets.com/2020/02/audio-data-analysis-deep-learning-python-part-1.html>)

We illustrate the use of `librosa`, a package for music and audio analysis.

Table of Contents:

[1. Load Audio File, Waveform and Spectrogram](#)

[2. Feature Extraction from Audio Signal](#)

[5. Summary: From Signal to Mel-Frequency Cepstral Coefficients \(MFCCs\)](#)

Preamble

```
In [2]: from scipy import signal
        from scipy.io import wavfile      # Load wave files
        import librosa                   # package for music and audio analysis
        import librosa.display
        import IPython.display as ipyd
        import matplotlib.pyplot as plt
        import numpy as np
```

1. Load Audio File, Waveform and Spectrogram

Load Audio File

`scipy.io`

We can use `scipy.io.wavfile.read()` to load an audio file into a time series and extract the sample rate:

```
sr, x = wavfile.read(filename)
```

The output `sr` is the sample rate and `x` is the time series as a numpy array.

`librosa`

We can use `librosa.load()` to load an audio file and extract the sample rate:

```
x, sr = wavfile.read(path, sr=22050, mono=True, offset=0.0, duration=None)
```

Arguments:

`sr` : sample rate. `None` means use the native sample rate of the file

`mono` : convert the signal to mono

`offset` : start reading after this time (in seconds)

`duration` : only load up to this much audio (in seconds)

Outputs:

`x` : time series as a numpy array. * Here we use the notations `x` and `y` interchangeably

`sr` : sample rate

Plotting waveform

```
librosa.display.waveshow(y, sr=22050, max_points=11025, x_axis='time', offset=0.0, ...)
```

Arguments:

`max_points` : Maximum number of samples to draw. If the number of raw samples in `y` exceeds `max_points`, the series is downsampled before the waveform is drawn.

`x_axis` : 'time' : values are plotted in second

`offset` : horizontal offset (in seconds) to start the waveform plot

Spectrogram

See DFT, STFT, Spectrograms and MFCCs.ipynb for a review on short-time Fourier transform (STFT).

1. Compute the STFT of the signal

```
stfft = librosa.stft(y, n_fft=2048, hop_length=None, ...)
```

2. Convert amplitude of STFT to decibel scale

```
stfttdB = librosa.amplitude_to_db(np.abs(stfft), ref=1.0, ...)
```

Arguments:

`ref` : the amplitude $\text{abs}(X)$ is scaled relative to `ref`: $20 \log_{10}(X/\text{ref})$. Zeros in the output correspond to positions where $S == \text{ref}$

3. Plot spectrogram

```
librosa.display.specshow(stfttdB, sr=22050, hop_length=512, fmin=None, fmax=None, ...)
```

Arguments:

`hop_length` : i.e. stride length of STFT in unit of number of samples

`x_axis`, `y_axis` : see documentation for options

<https://librosa.org/doc/main/generated/librosa.display.specshow.html?highlight=specshow>

<https://librosa.org/doc/main/generated/librosa.display.specshow.html?highlight=specshow>

Load Audio File

```
In [82]: # filename = 'violin_A.wav'
filename = 'bruckner.wav'

# use scipy.io.wavfile.read() to load audio file
# extract sample rate f_x and signal x
# sr, x = wavfile.read(filename)
# x = x / 1.0 # convert to float

# use librosa.load() to load audio file
# x, sr = librosa.load(filename, sr=None)
x, sr = librosa.load(filename, sr=None, offset=0.1)

# play audio on IPython
ipyd.Audio(rate=sr, data=y)
```

Out[82]:

▶ 0:05 / 0:05 ————— 🔊 ⋮

In [83]: x

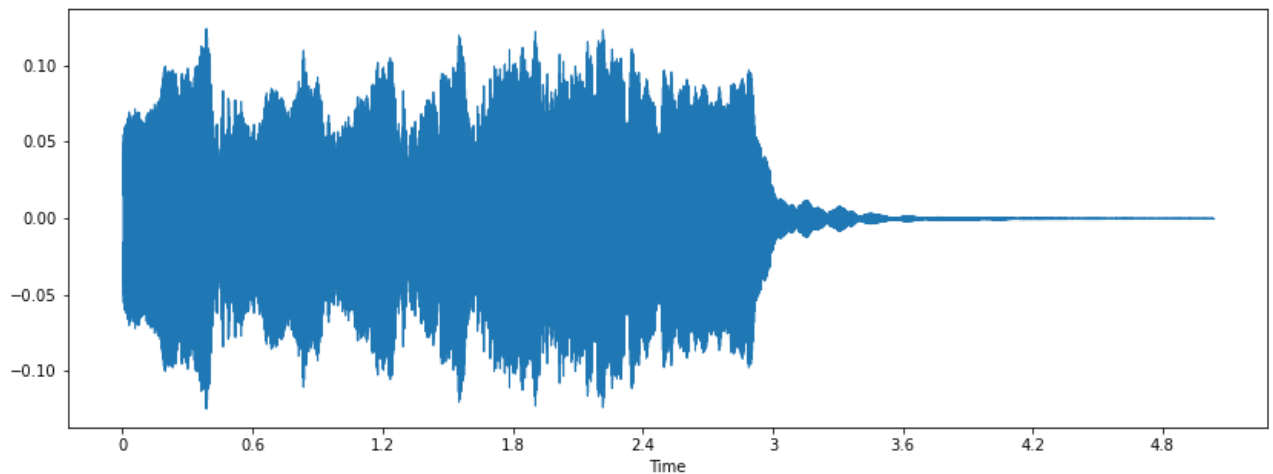
Out[83]: array([5.0354004e-03, 2.1362305e-04, -4.8828125e-03, ...,
-1.2207031e-04, 1.2207031e-04, -9.1552734e-05], dtype=float32)

In [84]: sr

Out[84]: 44100

Waveform plot

```
In [85]: plt.figure(figsize=(14, 5))
librosa.display.waveshow(x, sr=sr);
```

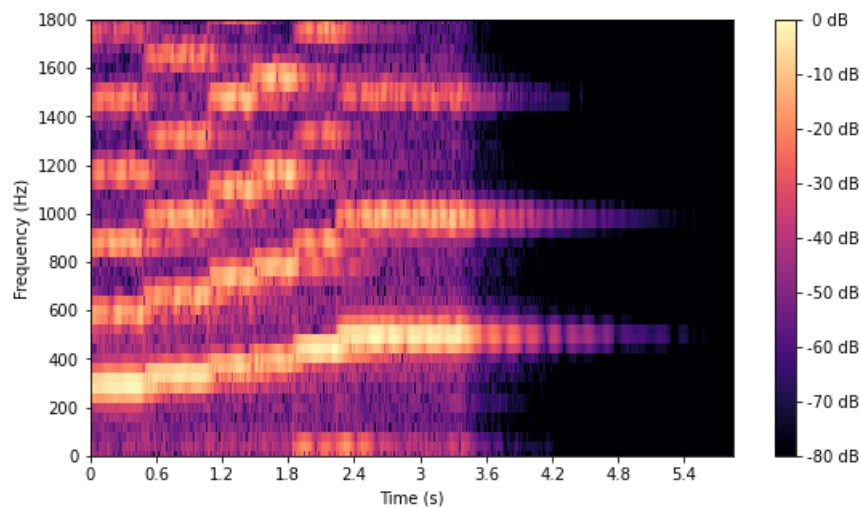


Spectrogram

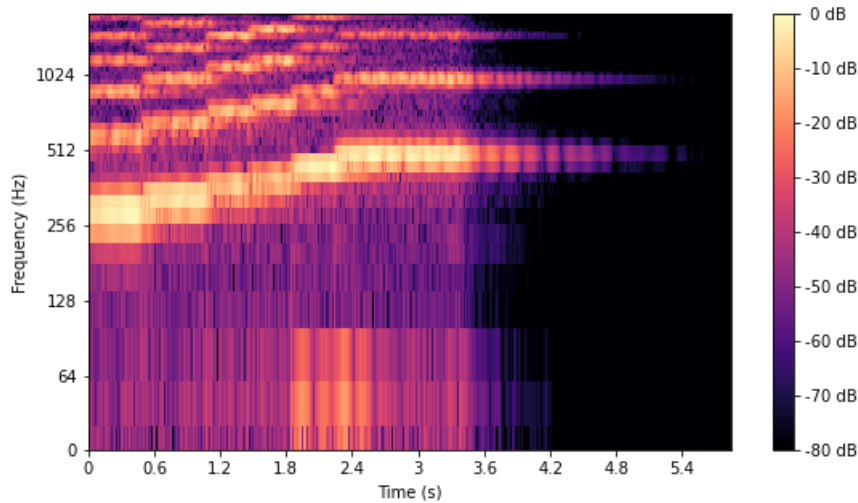
```
In [135]: # Short-time fast Fourier transform (STFFT)
n_fft = int(0.025*sr) # 25ms
hop_length = int(0.01*sr) # 10ms
stfft = librosa.stft(x, n_fft=n_fft, hop_length=hop_length)
abs_stfft = np.abs(stfft)

stftdB = librosa.amplitude_to_db(abs_stfft, ref=np.max)
```

```
In [136]: # Plot spectrogram
fig, ax = plt.subplots(figsize=(9,5))
img = librosa.display.specshow(stftdB, sr=sr,
                                y_axis='hz', x_axis='time', ax=ax, fmax=6000)
plt.xlabel('Time (s)')
plt.ylabel("Frequency (Hz)")
plt.ylim([0, 1800])
fig.colorbar(img, ax=ax, format="%2.0f dB");
```



```
In [137]: # Plot spectrogram, frequency in log scale
fig, ax = plt.subplots(figsize=(9,5))
img = librosa.display.specshow(stfttdB, sr=sr,
                               y_axis='log', x_axis='time', ax=ax, fmax=6000)
plt.xlabel('Time (s)')
plt.ylabel("Frequency (Hz)")
plt.ylim([0, 1800])
fig.colorbar(img, ax=ax, format="%2.0f dB");
```



In []:

2. Feature Extraction from Audio Signal

Useful function:

```
librosa.frames_to_time(frames, sr=22050, hop_length=512, n_fft=None)
```

converts frame counts to time (seconds).

In addition to the STFT which we visualise with a spectrogram, we want to extract useful features from the signal. Here we look at some **spectral features (frequency-based features)**:

1. Spectral Centroid

The **spectral centroid** f_c is defined as the mean of the frequencies present in the signal weighed by their magnitudes, aka the 'centre of mass' frequency of the signal:

$$f_c \equiv \frac{\sum_k |X_k| f_k}{\sum_k |X_k|} \quad (1)$$

It is implemented by

```
librosa.feature.spectral_centroid(y=None, sr=22050, n_fft=2048, hop_length=512, ...)
```

2. Spectral Rolloff

The **spectral rolloff** is the frequency below which a specified percentage (e.g. 85%) of the total spectral energy lies.

```
librosa.feature.spectral_rolloff(y=None, sr=22050, n_fft=2048, hop_length=512, roll_percent=0.85, ...)
```

3. Spectral Bandwidth

The **order- p spectral bandwidth** is defined as

$$\left(\sum_k S_k(t) \times (f_k(t) - \text{centroid}(t))^p \right)^{\frac{1}{p}}, \quad (2)$$

where $S_k(t)$ is the magnitude of the spectrogram.

```
librosa.feature.spectral_bandwidth(y=None, sr=22050, n_fft=2048, hop_length=512, p=2)
```

4. Zero-Crossing Rate

The **zero-crossing rate (ZCR)** is rate at which a signal changes from positive to zero to negative or from negative to zero to positive; given a signal x_t of length T ,

$$\text{ZCR} \equiv \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{I}\{x_t x_{t-1} < 0\}, \quad (3)$$

where $\mathbb{I}\{y\}$ is an **indicator function**: it is 1 if y is true and 0 otherwise.

The ZCR is usually higher for highly percussive sounds like those in metal and rock.

```
librosa.feature.zero_crossing_rate(y, frame_length=2048, hop_length=512, ...)
```

frame_length : I think it is the same as n_fft

5. Mel-Frequency Cepstral Coefficients(MFCCs)

The first dozen or so MFCCs concisely describe the overall shape of a spectral envelope, and model the characteristics of the human voice. See DFT, STFT, Spectrograms and MFCCs.ipynb for a review on the workflow of obtaining the MFCCs.

```
mfcc = librosa.feature.mfcc(y=None, sr=22050, S=None, n_mfcc=20, dct_type=2, norm='ortho', lifter=0,
**kwargs)
```

```
librosa.display.specshow(mfcc, sr=sr, x_axis='time');
```

6. Chroma feature

A chroma feature or vector is typically a 12-element feature vector indicating how much energy of each pitch class, {C, C#, D, D#, E, ..., B}, is present in the signal. In short, It provides a robust way to describe a similarity measure between music pieces.

```
chromagram = librosa.feature.chroma_stft(y=None, sr=22050, n_fft=2048, hop_length=512, tuning=None,
n_chroma=12, ...)
```

Returns a chromagram from a waveform or power spectrogram.

Arguments:

tuning : float or None . Deviation from A440 tuning in fractional chroma bins. If None , it is automatically estimated
n_chroma : Number of chroma bins to produce (12 by default)

```
librosa.display.specshow(chromagram, sr=sr, x_axis='time', y_axis='chroma', hop_length=hop_length,
cmap='coolwarm')
```

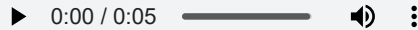
```
In [124]: # filename = 'violin_A.wav'
          filename = 'bruckner.wav'

          # use scipy.io.wavfile.read() to load audio file
          # extract sample rate f_x and signal x
          # sr, x = wavfile.read(filename)
          # x = x / 1.0 # convert to float

          # use librosa.load() to load audio file
          # x, sr = librosa.load(filename, sr=None)
          x, sr = librosa.load(filename, sr=None, offset=0.1)

          # play audio on IPython
          ipyd.Audio(rate=sr, data=y)
```

Out[124]:



```
In [135]: # Short-time fast Fourier transform (STFFT)
          n_fft = int(0.025*sr) # 25ms
          hop_length = int(0.01*sr) # 10ms
          stfft = librosa.stft(x, n_fft=n_fft, hop_length=hop_length)
          abs_stfft = np.abs(stfft)

          stftdB = librosa.amplitude_to_db(abs_stfft, ref=np.max)
```

```
In [125]: stfft.shape
```

Out[125]: (552, 504)

```
In [126]: t = librosa.frames_to_time(frames=np.arange(stfft.shape[1]), sr=sr,
          hop_length=hop_length, n_fft=n_fft)
```

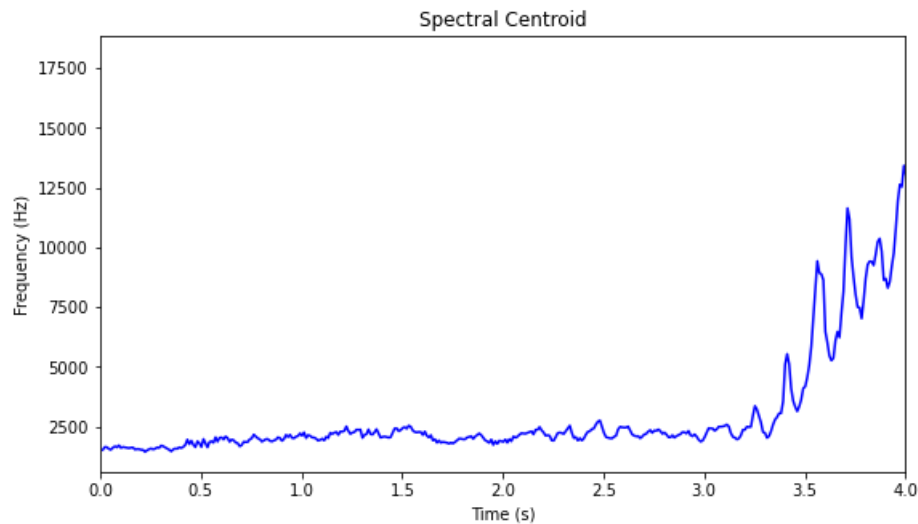
1. Spectral centroid

```
In [91]: spectral_centroids = librosa.feature.spectral_centroid(x, sr=sr, n_fft=n_fft, hop_length=hop_length)
```

```
In [92]: spectral_centroids.shape
```

Out[92]: (1, 504)

```
In [108]: plt.figure(figsize=(9,5))
plt.plot(t, spectral_centroids[0], color='b');
plt.xlim([0, 4])
# plt.ylim([1500, 18000])
plt.xlabel('Time (s)')
plt.ylabel("Frequency (Hz)")
plt.title('Spectral Centroid');
```



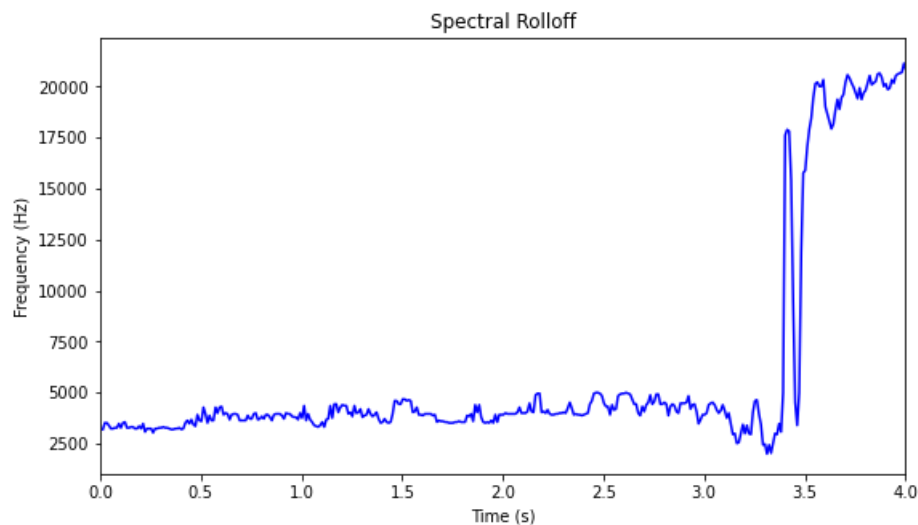
2. Spectral rolloff

```
In [98]: spectral_rolloff = librosa.feature.spectral_rolloff(x, sr=sr, n_fft=n_fft, hop_length=hop_length, roll_pe
```

```
In [100]: spectral_rolloff.shape
```

```
Out[100]: (1, 504)
```

```
In [107]: plt.figure(figsize=(9,5))
plt.plot(t, spectral_rolloff[0], color='b');
plt.xlim([0, 4])
# plt.ylim([1500, 18000])
plt.xlabel('Time (s)')
plt.ylabel("Frequency (Hz)")
plt.title('Spectral Rolloff');
```



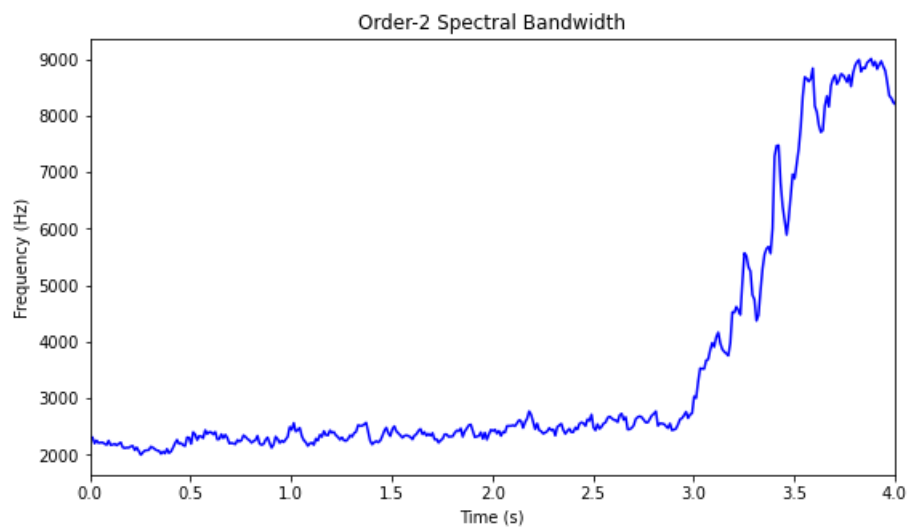
3. Order- p spectral bandwidth

```
In [102]: spectral_bandwidth_2 = librosa.feature.spectral_bandwidth(x, sr=sr,  
                                                                    n_fft=n_fft, hop_length=hop_length,  
                                                                    p=2)[0]
```

```
In [104]: spectral_bandwidth_2.shape
```

```
Out[104]: (504,)
```

```
In [106]: plt.figure(figsize=(9,5))  
plt.plot(t, spectral_bandwidth_2, color='b');  
plt.xlim([0, 4])  
# plt.ylim([1500, 18000])  
plt.xlabel('Time (s)')  
plt.ylabel("Frequency (Hz)")  
plt.title('Order-2 Spectral Bandwidth');
```



4. Zero-Crossing Rate (ZCR)

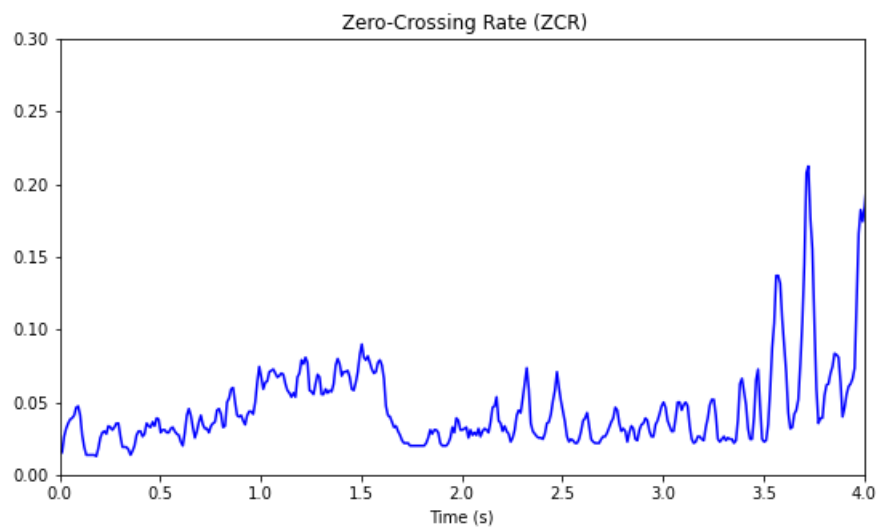
```
In [111]: zcr = librosa.feature.zero_crossing_rate(x, frame_length=n_fft, hop_length=hop_length)
```

```
In [112]: zcr.shape
```

```
Out[112]: (1, 504)
```



```
In [116]: plt.figure(figsize=(9,5))
plt.plot(t, zcr[0], color='b');
plt.xlim([0, 4])
plt.ylim([0, 0.3])
plt.xlabel('Time (s)')
# plt.ylabel("Frequency (Hz)")
plt.title(' Zero-Crossing Rate (ZCR)');
```



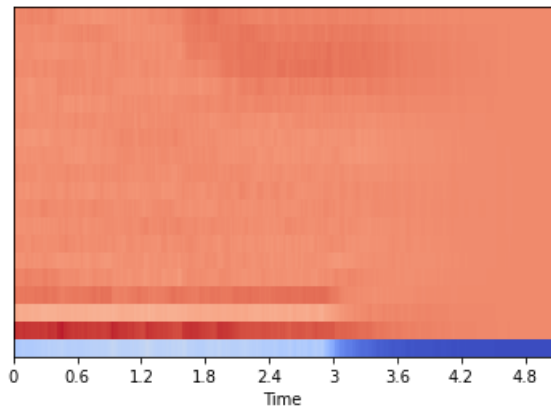
5. Mel-Frequency Cepstral Coefficients(MFCCs)

```
In [117]: mfcc = librosa.feature.mfcc(x, sr=sr, n_mfcc=20)
```

```
In [118]: mfcc.shape
```

```
Out[118]: (20, 434)
```

```
In [127]: # plot MFCCs
# plt.figure(figsize=(15, 7))
librosa.display.specshow(mfcc, sr=sr, x_axis='time');
```



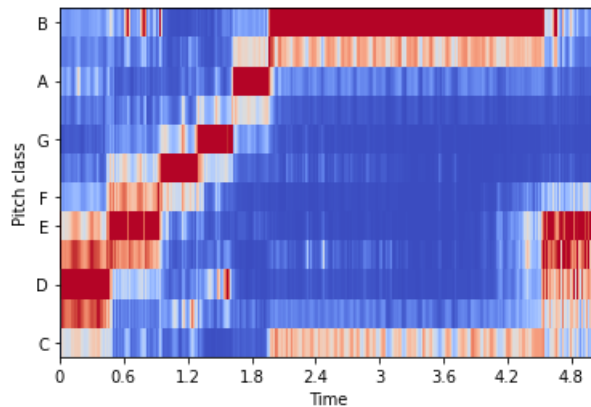
6. Chroma feature

```
In [141]: chromagram = librosa.feature.chroma_stft(x, sr=sr, n_fft=n_fft, hop_length=hop_length)
```

```
In [144]: chromagram.shape
```

```
Out[144]: (12, 504)
```

```
In [145]: # plot chromagram
librosa.display.specshow(chromagram, sr=sr, x_axis='time', y_axis='chroma',
                        hop_length=hop_length, cmap='coolwarm');
```



```
In [ ]:
```

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []: